Hafta 4 — Web Formlar

> **Ders:** Web Programlama · **Hafta:** 4 / 13 ders haftası · **Süre:** 3 × 50 dakika **Ön koşul:** Hafta 3 (Razor, görünüm, layout) ve Hafta 1 (HTTP isteği ve gövdesi) **Bu dosya sınav kapsamındadır.** Daha derin konular `H04-detay.md` dosyasındadır ve **sınav kapsamı dışıdır.**

🎯 Bu Haftanın Hedefleri

Bu haftanın sonunda:

1. Bir formun sunucuya **tam olarak ne gönderdiğini** — gövde biçimi, `Content-Type`, alan adları — **gösterebileceksiniz**.
2. `name` attribute'ünün neden tek belirleyici olduğunu **açıklayabileceksiniz**.
3. **GET/POST action çiftini** yazabilecek, `[HttpGet]` ve `[HttpPost]`'un ne işe yaradığını **söyleyebileceksiniz**.
4. **Model Binding**'in bir isteği action parametrelerine nasıl dönüştürdüğünü **açıklayabileceksiniz**.
5. Model Binding kaynaklarının **öncelik sırasını** (Form → Rota → Sorgu) **sıralayabilecek** ve `[FromQuery]` gibi attribute'lerle bu sırayı **değiştirebileceksiniz**.
6. Model Binding başarısız olduğunda ne olduğunu **gösterebilecek** ve `ModelState` ile gerçek hatayı **ayırt edebileceksiniz**.
7. **Post-Redirect-Get** desenini kurabilecek ve çözdüğü sorunu **anlatabileceksiniz**.
8. **Over-posting** saldırısını **gösterebilecek** ve **ViewModel** ile önleyebileceksiniz.

🧩 **Haftanın ilkesi — İstemciden gelen hiçbir veri güvenilir değildir**

> Formun HTML'ini siz yazarsınız; ama o HTML kullanıcının makinesinde çalışır ve kullanıcı onu değiştirebilir. Sunucuya ulaşan şey sizin yazdığınız form değil, **kullanıcının göndermeyi seçtiği veridir.** Bu hafta öğrendiğimiz her araç, aradaki bu boşluğu kapatmak içindir.

---

1. Form ve HTTP

Hafta 3'te veri **tek yöne** aktı: sunucudan kullanıcıya. Controller veriyi hazırladı, Razor HTML üretti, tarayıcı çizdi. Bu hafta **ters yöne** geçiyoruz.

1.1. Önce soru: tarayıcı ne gönderiyor?

Hafta 1'de bir HTTP isteğinin **başlıkları** ve bir **gövdesi** olduğunu görmüştük. Bir form "Gönder" dendiğinde tarayıcı o gövdeye ne koyar?

Şu formu ele alalım:

```html
<form method="post" action="/Form">
    <input type="text"   id="Baslik"   name="Baslik" value="Suc ve Ceza" />
    <input type="text"   id="Yazar"    name="Yazar"  value="Dostoyevski" />
    <input type="number" id="Fiyat"    name="Fiyat"  value="120" />
    <input type="text"   id="notAlani" name="Notlar" value="deneme" />
    <input type="text"   id="unutulan"               value="bu değer gitmeyecek" />
    <button type="submit">Gönder</button>
</form>
```

Sunucuya ulaşan istek:

```http
POST /Form HTTP/1.1
Content-Type: application/x-www-form-urlencoded

Baslik=Suc+ve+Ceza&Yazar=Dostoyevski&Fiyat=120&Notlar=deneme
```

Buradan **dört** sonuç çıkar:

Gözlem	Sonucu
Gövde `ad=değer&ad=değer` biçiminde	Bu biçimin adı `Content-Type` satırında yazıyor: **`application/x-www-form-urlencoded`**
Boşluk `+`, özel karakterler `%XX` oldu	Gövde **URL kodlamasından** geçer; çözme işini çatı yapar
Gönderilen ad **`name`**	`Baslik`, `Yazar`, `Fiyat`, `Notlar` — hiçbiri `id` değil
`name`'i olmayan alan **gövdede yok**	`unutulan` alanının değeri sunucuya **hiç ulaşmadı**

> 🧪 Bunu kendiniz görün: `KampusKitap` projesinde **`/Form`** sayfasını açın, formu gönderin. Sayfa, isteğin gövdesini **olduğu gibi** ekrana basıyor. Ayrıca tarayıcıda **F12 → Network → Payload** ile de aynı şeyi görebilirsiniz.

1.2. `name` — sunucunun gördüğü tek ad

Bir `<input>` elemanının üç adı olabilir ve **üçü farklı işler yapar**:

Attribute	Kim kullanır	Sunucuya gider mi
`id`	CSS, JavaScript, `<label for="…">`	❌ hayır
`class`	CSS	❌ hayır
`name`	**HTTP** — gövdedeki anahtar	✅ **evet**

> 🐛 **Dönem boyunca en sık duyacağınız cümle:** *"Formu gönderdim ama sunucuda veri boş geliyor."* Sebebi neredeyse her zaman ikisinden biridir: `name` yok/yanlış yazılmış, ya da `method="post"` unutulmuş.

İşaretlenmemiş onay kutusu gövdede bulunmaz. `false` diye bir şey gönderilmez; alan **hiç yoktur**. Bu, HTML'in davranışıdır, ASP.NET Core'un değil. (Hafta 5'te Tag Helper'ların bu sorunu nasıl çözdüğünü göreceğiz.)

1.3. GET ile POST arasındaki fark

```
<form method="get"  action="/Kitap/Ara">   <!-- değerler ADRESE gider -->
<form method="post" action="/Kitap/Ekle">  <!-- değerler GÖVDEYE gider -->
```

	GET	POST
Veri nereye	**Adres çubuğuna** (query string)	**İstek gövdesine**
Yer imine eklenir mi	✅ evet	❌ hayır
Tarayıcı geçmişinde görünür mü	✅ evet (parola da!)	❌ hayır
Yenilenince ne olur	Zararsız tekrarlanır	**Tekrar gönderilir** (bölüm 3.1)
Uzunluk sınırı	Pratikte var (adres uzunluğu)	Pratikte yok
Ne zaman	**Okuma**: arama, filtre, sayfalama	**Değiştirme**: kayıt, güncelleme, silme

> **Kural:** Sunucuda bir şeyi **değiştiren** her işlem POST olmalıdır. Arama formu GET, kayıt formu POST.

1.4. GET/POST action çifti

Bir form sayfası **iki** action ister: biri formu gösterir, diğeri gelen veriyi işler. İkisinin **adı aynı**, metotları farklıdır.

```
// 1) Formu göster — GET /Kitap/Ekle
[HttpGet]
public IActionResult Ekle()
{
    return View();
}

// 2) Gelen veriyi işle — POST /Kitap/Ekle
[HttpPost]
public IActionResult Ekle(Kitap kitap)
{
    _depo.Ekle(kitap);
    return RedirectToAction("Index");   // bölüm 3.2 — PRG
}
```

Aynı adı taşıyan iki metot **aşırı yükleme (overload)** olduğu için parametreleri farklı olmalıdır — zaten öyle: biri parametresiz, diğeri modeli alıyor.

1.5. `[HttpGet]` ve `[HttpPost]`

Bu attribute'ler action'ın **hangi HTTP metoduna** cevap vereceğini söyler.

Yazım	Sonuç
`[HttpGet]`	Yalnızca GET isteklerinde çalışır
`[HttpPost]`	Yalnızca POST isteklerinde çalışır
Hiçbiri yok	**Her metoda** cevap verir

Hafta 2'den beri action'larımızda attribute yoktu ve çalışıyorlardı — çünkü attribute yokluğu "hepsini kabul et" demektir. Form çiftinde ise **zorunlu**: ikisi de aynı adı taşıdığı için çatı hangisini seçeceğini bilemez ve `AmbiguousMatchException` fırlatır.

> ⚠️ Bir action'ı yalnızca `[HttpPost]` yapmak onu **güvenli yapmaz**. Yalnızca adres çubuğuna yazılarak çağrılmasını engeller. Güvenlik Hafta 11'in konusu.

1.6. `IFormCollection` — ham okuma

Gelen form verisine, model binding'e hiç uğramadan da erişilebilir:

```
[HttpPost]
public IActionResult Ekle(IFormCollection form)
{
    var baslik = form["Baslik"];          // StringValues
    var fiyat  = int.Parse(form["Fiyat"]!); // elle çevirmek zorundasınız
    …
}
```

Çalışır. **Ama tercih edilmez:**

Sorun	Neden
Tip güvenliği yok	Her şey metin; her alanda elle `Parse`
Yazım hatası sessiz	`form["Balsik"]` boş döner, derleyici uyarmaz
Doğrulama yok	`ModelState` dolmaz → Hafta 6'nın araçları çalışmaz
Okunmaz	10 alanlı formda 10 satır dönüştürme kodu

> Ne zaman meşru: alan adları **çalışma zamanında** belli oluyorsa (örneğin kullanıcının tanımladığı dinamik anket soruları). Onun dışında model binding kullanın.

---

2. Model Binding

2.1. Model Binding nedir?

Hafta 2'de action parametrelerinin "bir yerlerden" geldiğini görmüştük. O "bir yer"in adı **Model Binding**'dir.

> **Model Binding**, gelen HTTP isteğindeki **metin** değerleri bulup action parametrelerinin **tiplerine** çevirerek dolduran mekanizmadır.

Çatı şunu yapar:

1. Action'ın parametrelerine bakar: `Ekle(Kitap kitap)` → bir `Kitap` lazım.
2. `Kitap` sınıfının özelliklerine bakar: `Baslik`, `Yazar`, `Fiyat`, …
3. Her özellik için **istekte aynı adda bir değer arar**.
4. Bulduğu metni özelliğin **tipine çevirir** (`"120"` → `int 120`).
5. Çeviremezse **hata fırlatmaz**; varsayılan değeri bırakır ve `ModelState`'e not düşer (bölüm 2.7).

Bu yüzden form alanının `name`'i ile sınıfın özellik adı **birebir aynı** olmalıdır.

2.2. Model Binding kaynakları ve öncelik sırası

Çatı değeri üç yerde arar. Aynı ad birden fazla yerde varsa **sıra önemlidir**:

Sıra	Kaynak	Örnek
1	**Form gövdesi**	POST gövdesindeki `deger=FORM`
2	**Route data**	`/Form/Kaynak/{deger}` kalıbındaki segment
3	**Query string**	`?deger=SORGU`

İlk bulan kazanır; sonrakilere bakılmaz.

```
POST /Form/Kaynak/ROTA?deger=SORGU
Gövde: deger=FORM

→ action parametresine gelen değer: FORM
```

> ⚠️ **Bu sırayı ters ezberlemek çok yaygındır.** Adres çubuğunda gördüğümüz için query string'in öncelikli olduğunu sanırız; **en düşük öncelikli olan odur.**
>
> 🧪 Projede `/Form/Kaynak/ROTA?deger=SORGU` sayfasını açın. Sayfa üç kaynağın değerini ayrı ayrı gösteriyor ve hangisinin kazandığını yazıyor. POST düğmesine basınca üçü birden yarışıyor.

Dört durumun hepsi:

Rota	Sorgu	Form	Kazanan
ROTA	—	—	**ROTA**
ROTA	SORGU	—	**ROTA**
—	SORGU	—	**SORGU**
ROTA	SORGU	FORM	**FORM**

2.3. Basit tip bağlama

`int`, `string`, `bool`, `DateTime`, `decimal` gibi tipler doğrudan bağlanır:

```
public IActionResult Ara(string? kelime, int sayfa = 1)
```

`/Kitap/Ara?kelime=roman&sayfa=2` → `kelime = "roman"`, `sayfa = 2`.

Varsayılan değer (`= 1`) parametre **hiç gelmezse** kullanılır.

2.4. Karmaşık tip bağlama

Parametre bir sınıfsa çatı **özellik özellik** doldurur:

```
public class Kitap
{
    public string Baslik { get; set; } = "";
    public string Yazar  { get; set; } = "";
    public decimal Fiyat { get; set; }
}

[HttpPost]
public IActionResult Ekle(Kitap kitap) { … }
```

Form alanları:

```
<input name="Baslik" />
<input name="Yazar" />
<input name="Fiyat" />
```

İç içe nesnede ad **noktayla** yazılır:

```
public class Siparis { public Adres Teslimat { get; set; } }
```

```
<input name="Teslimat.Sehir" />
```

2.5. Koleksiyon bağlama

Birden çok değer için **köşeli parantezli indis** kullanılır:

```
<input name="Etiketler[0]" value="roman" />
<input name="Etiketler[1]" value="klasik" />
```

```
public List<string> Etiketler { get; set; } = [];
```

Aynı `name`'i tekrarlamak da çalışır (`<input name="Etiketler" />` ×2) ama indisli yazım nesne listelerinde zorunludur:

```
<input name="Kitaplar[0].Baslik" />
<input name="Kitaplar[0].Fiyat" />
<input name="Kitaplar[1].Baslik" />
```

> ⚠️ İndisler **0'dan başlamalı ve boşluk olmamalıdır**. `[0]`, `[2]` gönderirseniz çatı `[1]`'de durur ve listeye yalnızca ilk eleman girer.

2.6. Kaynağı zorlamak: `[From…]` attribute'leri

Varsayılan sırayı devre dışı bırakıp kaynağı **siz** seçebilirsiniz:

Attribute	Nereden okur
`[FromForm]`	Yalnızca form gövdesinden
`[FromRoute]`	Yalnızca route data'dan
`[FromQuery]`	Yalnızca query string'den
`[FromBody]`	İstek gövdesinden **JSON olarak** — Web API, Hafta 12
`[FromHeader]`	İstek başlığından
`[FromServices]`	DI kabından (Hafta 2) — istekten değil

```
public IActionResult Kaynak([FromQuery] string? deger)
```

Aynı istek, aynı üç kaynak — ama artık kazanan **SORGU**:

```
POST /Form/KaynakZorla/ROTA?deger=SORGU   (gövde: deger=FORM)
→ attribute yokken: FORM
→ [FromQuery] ile : SORGU
```

> `[FromBody]` bir action'da **yalnızca bir kez** kullanılabilir: gövde tek bir akıştır, iki kez okunamaz.

2.7. Model Binding başarısız olduğunda

Bu bölüm bu haftanın en çok hata ürettiren yeridir.

```
public IActionResult Bagla(int sayi, int? sayiNullable, DateTime tarih)
```

`/Form/Bagla?sayi=abc&sayiNullable=abc&tarih=xyz` adresini açınca:

Parametre	Gelen değer

`int sayi`	**0**
`int? sayiNullable`	**null**
`DateTime tarih`	**0001-01-01T00:00:00**

Hata sayfası görmezsiniz. İstisna fırlamaz. Sayfa açılır.

Çatı, çeviremediği her değer için parametreye **tipin varsayılanını** koyar ve durumu `ModelState`'e yazar:

```
ModelState.IsValid   // false
```

🧩 **Dersin can alıcı noktası:** `sayi` artık **0**. `if (sayi == 0)` yazarsanız "*kullanıcı 0 girdi*" ile "*kullanıcı saçma bir şey girdi*" durumlarını **ayırt edemezsiniz**. Ayrımı yapan tek şey `ModelState`'tir.

```
[HttpPost]
public IActionResult Ekle(Kitap kitap)
{
    if (!ModelState.IsValid)
        return View(kitap);        // formu hatalarla birlikte geri göster

    _depo.Ekle(kitap);
    return RedirectToAction("Index");
}
```

➡️ SOON `ModelState`'in asıl gücü **Hafta 6**'da (Model Doğrulama) açılacak. Bu hafta yalnızca varlığını ve neden gerektiğini biliyoruz.

---

3. PRG Deseni ve Form Güvenliğinin Temeli

3.1. Problem: POST'tan sonra yenileme

Kullanıcı formu gönderdi, action kaydı ekledi ve doğrudan bir görünüm döndürdü:

```
[HttpPost]
public IActionResult Ekle(Kitap kitap)
{
    _depo.Ekle(kitap);
    return View("Tamam");     // ⚠️ adres hâlâ POST /Kitap/Ekle
}
```

Kullanıcı **F5**'e basar. Tarayıcı son isteği tekrarlar — ve o istek bir **POST**'tur. Tarayıcı "*Formu yeniden göndermek istiyor musunuz?*" diye sorar; kullanıcı "Evet" derse **kayıt ikinci kez eklenir**.

Geri/ileri tuşları da aynı sorunu üretir. Bu, kullanıcının hatası değil; adres çubuğunda bir **POST** bırakmanın doğal sonucudur.

3.2. Çözüm: Post-Redirect-Get

Desen üç adımdır ve adını adımlarından alır:

#	Adım	Ne olur
P	**Post**	Form gönderilir, sunucu işi yapar
R	**Redirect**	Sunucu **302** ile "*şu adrese git*" der
G	**Get**	Tarayıcı o adrese **GET** yapar; adres çubuğunda artık GET vardır

```
[HttpPost]
public IActionResult Ekle(Kitap kitap)
{
    if (!ModelState.IsValid) return View(kitap);

    _depo.Ekle(kitap);
    return RedirectToAction("Index");   // ← PRG'nin R'si
}
```

Artık F5 yalnızca listeyi yeniden **okur**. Hiçbir şey tekrar eklenmez.

> **Kural:** Veri **değiştiren** bir POST action'ı, başarı durumunda `View()` değil `RedirectToAction()` döndürmelidir. Başarısızlıkta (`ModelState` geçersiz) `View()` döner — çünkü kullanıcının düzeltmesi gereken form oradadır.

3.3. Over-posting

Model binding **adı uyan her alanı** doldurur. Bu, işimize yaradığı kadar başımıza da iş açar.

```
public class Kitap
{
    public string Baslik { get; set; } = "";
    public decimal Fiyat { get; set; }
    public bool Onayli { get; set; }     // ⚠️ yalnızca yönetici değiştirmeli
}

[HttpPost]
public IActionResult Ekle(Kitap kitap) { … }   // ⬬ entity doğrudan parametre
```

Formunuzda `Onayli` diye bir alan **yok**. Ama kullanıcı isteği kendi oluşturabilir:

```
POST /Kitap/Ekle
Content-Type: application/x-www-form-urlencoded

Baslik=X&Fiyat=1&Onayli=true
```

Model binding `Onayli`'yı **seve seve** doldurur. Kullanıcı kendi kaydını onaylamış olur.

Bu saldırıya **over-posting** ya da **mass assignment ** denir. Gerçek bir uygulamada alan `IsAdmin`, `Rol`, `Bakiye` ya da `KullaniciId` olabilir.

> 🧪 Projede `/Form/Tuzak` sayfasında iki form var. Tek fark ikincisindeki fazladan satır. Birincide `Onayli = False`, ikincide `True`.

3.4. Çözüm: ViewModel

Formun alabileceği alanları **açıkça** tanımlayan ayrı bir sınıf yazın:

```
// ViewModels/KitapEkleVM.cs — formun alabileceği alanlar, sadece bunlar
public class KitapEkleVM
{
    public string Baslik { get; set; } = "";
    public string Yazar  { get; set; } = "";
    public decimal Fiyat { get; set; }
    // Onayli YOK — bağlanacak bir özellik olmadığı için gönderilse de düşer
}

[HttpPost]
public IActionResult Ekle(KitapEkleVM girdi)
{
    if (!ModelState.IsValid) return View(girdi);

    var kitap = new Kitap
    {
        Baslik = girdi.Baslik,
        Yazar  = girdi.Yazar,
        Fiyat  = girdi.Fiyat,
        Onayli = false            // ← kararı SİZ veriyorsunuz
    };
    _depo.Ekle(kitap);
    return RedirectToAction("Index");
}
```

ViewModel'in iki faydası:

1. Bağlanabilecek alanlar listesi **sonludur ve görünürdür**.
2. Entity'ye yeni bir hassas alan eklediğinizde form **otomatik olarak** onu kabul etmez.

> `[Bind("Baslik,Yazar,Fiyat")]` attribute'ü de aynı işi yapar ama **kırılgandır**: entity'ye alan eklendiğinde listeyi güncellemeyi unutursunuz ve unutmak **hata vermez**. ViewModel bu riski taşımaz.

3.5. Gizli alanlara güvenmemek

```
<input type="hidden" name="Fiyat" value="120" />
```

"Gizli" sözcüğü aldatıcıdır: alan kullanıcıdan **gizlenmiştir**, korunmamıştır. Kullanıcı sayfanın kaynağını görebilir, değeri değiştirip gönderebilir.

> 🧩 **Haftanın ilkesi burada tam anlamını buluyor:** *Formun HTML'i sizin, gönderilen veri değil.* Fiyat, indiriminiz, kullanıcı kimliği gibi değerler **sunucuda** belirlenmelidir — istemciden gelen değere değil, kendi kayıtlarınıza bakarak.

3.6. Dosya yükleme — bu haftanın sınırı

Dosya yükleyen formda iki şey değişir:

```
<form method="post" enctype="multipart/form-data">
    <input type="file" name="Kapak" />
</form>
```

```
public IActionResult Yukle(IFormFile kapak) { … }
```

`enctype="multipart/form-data"` olmadan dosya **gönderilmez**; gövde yine `application/x-www-form-urlencoded` olur ve yalnızca dosya **adı** gider.

> ➡️ Dosya yüklemenin tamamı — boyut sınırı, tip denetimi, güvenli saklama — **Hafta 13**'ün konusudur. Bu hafta yalnızca bu iki satırı tanıyoruz.

🐛 Hata Köşesi — Hafta 4

Belirti	Gerçek sebep	Çözüm
Form gönderildi, model **boş** geldi	`name` attribute'ü yok ya da sınıftaki özellik adıyla uyuşmuyor	F12 → Network → Payload ile gövdeyi okuyun; adları karşılaştırın
Form gönderiyorum ama **GET**'e düşüyor	`method="post"` unutulmuş	`<form method="post" …>`
`AmbiguousMatchException`	Aynı adda iki action var, `[HttpGet]` / `[HttpPost]` konmamış	Attributeleri ekleyin
Sayısal alan hep **0** geliyor	Değer çevrilemiyor (boş, metin, ondalık ayracı)	`ModelState.IsValid`'i kontrol edin; 0 ile hatayı karıştırmayın
Onay kutusu hep **false**	İşaretlenmemiş kutu gövdede **hiç gelmez**	Normaldir; Tag Helper gizli alan ekler (Hafta 5)
Query stringndeki değer **göz ardı ediliyor**	Aynı ad formda da var; **form öncelikli**	`[FromQuery]` ile kaynağı zorlayın
F5'te kayıt **ikinci kez** ekleniyor	POST sonrası `View()` döndürülmüş	`RedirectToAction()` — PRG (bölüm 3.2)

Dosya yüklenmiyor, adı geliyor	`enctype="multipart/form-data"` yok	Forma `enctype` ekleyin
Liste yalnızca **ilk** elemanı alıyor	İndisler atlamalı: `[0]`, `[2]`	İndisler 0'dan başlamalı ve kesintisiz olmalı

🕳️ Klasik Tuzak — Entity'yi doğrudan action parametresi yapmak

```
[HttpPost]
public IActionResult Ekle(Kitap kitap)     // ⚠️ veritabanı varlığı, doğrudan
{
    _depo.Ekle(kitap);
    return RedirectToAction("Index");
}
```

Neden tuzak: Kod çalışır, testler geçer, form doğru görünür. Hiçbir aşamada hata alınmaz. Açık yalnızca **birisi aramaya başladığında** ortaya çıkar — ve o kişi sizden önce davranır.

Üstelik kusur **büyüyerek** gelir: entity'ye bugün eklediğiniz zararsız bir alan (`Indirim`, `Puan`, `Rol`), yarın formun kabul ettiği bir alana dönüşür ve bunu **size kimse haber vermez**.

Doğrusu: formun alabileceği alanları bir **ViewModel**'de açıkça yazın, entity'yi sunucuda **siz** oluşturun (bölüm 3.4).

> Aynı kalıbı üçüncü kez görüyoruz: Hafta 2'de middleware sırası, Hafta 3'te görünüm içindeki sorgu, bu hafta over-posting. Üçü de **hata vermiyor**. Bu dersin en pahalı hataları, çalışan koddan çıkıyor.

📖 Terimler Sözlüğü

Terim	Ne demek
Form	Kullanıcıdan veri toplayıp sunucuya gönderen HTML yapısı
`name`	Form alanının HTTP gövdesindeki anahtarı
`application/x-www-form-urlencoded`	Varsayılan form gövde biçimi: `ad=değer&ad=değer`
`multipart/form-data`	Dosya içeren formların gövde biçimi
Model Binding	İstek verisini action parametrelerine dönüştüren mekanizma
Value provider	Belirli bir kaynaktan (form, rota, sorgu) değer okuyan bileşen
`ModelState`	Model Binding ve doğrulama sonuçlarını tutan nesne
PRG	POST sonrası yönlendirme deseni
Over-posting / Mass assignment	Kullanıcının beklenmeyen alanları doldurması
ViewModel	Yalnızca bir ekranın ihtiyacını taşıyan sınıf

Entity	Veri katmanındaki asıl nesne (Hafta 8'de veritabanına bağlanacak)

✅ Kontrol Soruları

A. Form ve HTTP

1. `<input type="text" id="adSoyad" name="Ad" value="Ayşe">` alanı gönderildiğinde gövdede hangi anahtar görünür?
2. Bir `<input>` elemanının `name` attribute'ü yoksa ne olur?
3. İşaretlenmemiş bir onay kutusu gövdeye ne gönderir?
4. Varsayılan form gövdesinin `Content-Type` değeri nedir?
5. Arama formu GET mi POST mu olmalı? Kayıt formu? Neden?
6. Aynı adı taşıyan iki action'dan `[HttpGet]` / `[HttpPost]` kaldırılırsa ne olur?

B. Model Binding

7. Model Binding'in beş adımını sırayla yazın.
8. Model Binding kaynaklarının **öncelik sırası** nedir?
9. `POST /K/5?deger=S` isteğinde gövdede de `deger=F` varsa parametreye ne gelir?
10. Aynı istekte parametreye `[FromQuery]` eklenirse ne gelir?
11. `Teslimat.Sehir` adlı bir form alanı hangi C# yapısına bağlanır?
12. `int sayi` parametresine `abc` gelirse `sayi` kaç olur? `int? sayi` olsaydı?
13. 12. sorudaki durumda `ModelState.IsValid` ne döner? Bu neden önemli?
14. `[FromBody]` bir action'da neden yalnızca bir kez kullanılabilir?

C. PRG ve güvenlik

15. POST sonrası `View()` döndürmenin somut zararı nedir?
16. PRG'nin üç adımını ve her adımdaki HTTP metodunu yazın.
17. `ModelState` geçersizken PRG uygulanır mı? Neden?
18. Over-posting nedir? Bir örnekle anlatın.
19. `[Bind("…")]` yerine ViewModel önerilmesinin sebebi nedir?
20. "Gizli alan" neden güvenlik önlemi değildir?
21. Dosya yükleyen formda hangi iki şey değişir?

📑 Kaynakça

> Her kaynağın yanında **bu ders notunun hangi bölümünü** dayandırdığı yazılıdır. Bağlantıların sonundaki `?view=aspnetcore-10.0` **kasıtlıdır**: sürüm sabitlenmezse arama motoru sizi eski sürüme düşürür. **Örnekte `Startup.cs` görüyorsanız yanlış sürümdesiniz.**

Zorunlu okuma

Kaynak	Dayandırdığı bölüm
Model Binding in ASP.NET Core — https://learn.microsoft.com/aspnet/core/mvc/models/model-binding?view=aspnetcore-10.0	Bölüm 2 tamamı — kaynaklar, öncelik sırası, basit/karmaşık/koleksiyon bağlama, `[From…]`, Model Binding hatası
Routing to controller actions — https://learn.microsoft.com/aspnet/core/mvc/controllers/routing?view=aspnetcore-10.0	Bölüm 1.4, 1.5 — `[HttpGet]` / `[HttpPost]` ve action seçimi
Tag Helpers in forms in ASP.NET Core — https://learn.microsoft.com/aspnet/core/mvc/views/working-with-forms?view=aspnetcore-10.0	Bölüm 1.2, 3.6 — form alanları, onay kutusu davranışı, `enctype` (**Hafta 5 köprüsü**)
Sending form data (MDN) — https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Forms/Sending_and_retrieving_form_data	Bölüm 1.1, 1.2, 1.3 — gövde biçimi, `name`, GET/POST farkı

Güvenlik ve desen

Kaynak	Dayandırdığı bölüm
Implement CRUD — ASP.NET MVC with EF Core, "Overposting" başlığı — https://learn.microsoft.com/aspnet/core/data/ef-mvc/crud?view=aspnetcore-10.0#overposting	Bölüm 3.3, 3.4 — over-posting ve ViewModel çözümü (sayfanın tamamı **Hafta 8** konusudur; yalnızca bu başlığı okuyun)
Post/Redirect/Get (desen tanımı) — https://en.wikipedia.org/wiki/Post/Redirect/Get	Bölüm 3.1, 3.2 — desenin adı ve gerekçesi
HTTP POST (MDN) — https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Methods/POST	Bölüm 1.3 — POST'un tanımı ve yenileme davranışı

Köprüler (sonraki haftalara)

Kaynak	Dayandırdığı bölüm
Model validation in ASP.NET Core MVC — https://learn.microsoft.com/aspnet/core/mvc/models/validation?view=aspnetcore-10.0	Bölüm 2.7 — `ModelState`'in asıl kullanımı (**Hafta 6**)
Upload files in ASP.NET Core — https://learn.microsoft.com/aspnet/core/mvc/models/file-uploads?view=aspnetcore-10.0	Bölüm 3.6 — dosya yükleme (**Hafta 13**)

📚 Daha Fazlası

`H04-detay.md` dosyasında (**sınav kapsamı dışı**):

- Value providerların (value provider) sırası nereden geliyor, nasıl değiştirilir
- Custom model binder (custom model binder) yazmak

- `TryUpdateModelAsync` ve `[Bind]` ile seçici bağlama
- `StringValues` nedir, `IFormCollection` neden onu döndürür
- Kültüre bağlı çeviri: ondalık ayracı ve tarih biçimi sorunu
- `TempData` ile PRG sonrası mesaj taşımak (Hafta 7 ön tadı)
- CSRF'nin ne olduğu ve neden Hafta 5'e bırakıldığı

➡️ Haftaya Hazırlık

Bu hafta formu **elle** yazdık: her `<input>`'un `name`'ini kendimiz koyduk, sınıftaki özellik adıyla eşleştiğinden kendimiz emin olduk. Bu eşleşme **bizim sorumluluğumuzdaydı** ve yazım hatası yaptığımızda kimse uyarmadı.

Hafta 5'te bu işi çatıya devrediyoruz: **Tag Helper'lar**.

Düşünerek gelin: `name="Baslik"` yazmak yerine çatıya *"bu alan modelin `Baslik` özelliği için"* diyebilseydik ne kazanırdık? Yazım hatası **ne zaman** yakalanırdı?

> 🏠 Ev göreviniz bu sorunun cevabını hazırlıyor: elle kurduğunuz formun ürettiği HTML'i **saklayın**. Hafta 5'te Tag Helper'ın ürettiğiyle yan yana koyacağız.